CityTech TTP Summer Bootcamp

Project Day: College Schedule Builder

2026-06-11

Agenda

1. Review
2. Commit Habits and Branches
3. Starting a Project + Repo
4. Continue: College Schedule Builder

Review

What are the most important things you remember from yesterday?

Commit Early, Commit Often

Make **small, frequent commits** as you work - it's a core habit and a real safety net.

- **Save points**: roll back to a working version when something breaks
- **A clear history**: each commit tells the story of what changed and why
- **Smaller diffs**: easier to review, debug, and find where a bug crept in
- **Less to lose**: every push backs your work up on GitHub

Write short, meaningful messages: *what* changed and *why*.

A Peek Ahead: Branches

Just priming for now - we'll dig into this later.

- A **branch** is a separate line of work - make changes without affecting `main`
- Lets you try ideas (or let teammates work in parallel) and then **merge** back
- **For today, working on `main` is fine**: full branching and merging comes in a future session

`master` vs. `main`

Same idea - it's just the name of the **default branch**.

- `master` : git's old default branch name
- `main` : the modern default (GitHub switched new repos to `main` in 2020, as more inclusive terminology)
- **Functionally identical**: the default branch can be named anything; only the name changed
- You'll still see `master` in older repos and tutorials; new repos use `main`

Starting a Project + Repo (do it in this order)

First, make `main` your default branch (once per machine):

```
git config --global init.defaultBranch main
```

Then - in this order, to avoid GitHub's race condition:

1. **Create the repo on GitHub** - leave it **empty** (no README)
2. **Clone** the empty repo, then `cd` into it
3. **Create the boilerplate inside it** - don't nest a new folder

```
git clone git@github.com:you/college-schedule-builder.git
cd college-schedule-builder
yarn create vite . --template react-ts   # "." scaffolds into THIS folder
```

> `.` scaffolds into the current folder (no nested directory). Already on `master`?

"Unrelated Histories"

You may hit: `fatal: refusing to merge unrelated histories`.

- It means two repos have **separate commit histories** with **no shared starting point**
- Common cause: a GitHub repo that **already has commits** (e.g. you initialized it with a README) plus a **separately created** local repo - git won't auto-merge them
- This is exactly why we **create an empty repo and clone it first** (previous slide)

If you're already stuck with it, you can force the merge:

```
git pull origin main --allow-unrelated-histories
```

...but the clean workflow means you shouldn't need this.

Continue Working on College Schedule Builder

Spend today building.

- Aim for the **Create + Read** requirements first
- **Commit early and often** as you go
- Then take on the **Tiered Learning** stretch goals
- Full spec: `project_specs/college_schedule_builder.md`